ACOL 216: Introduction to Computer Architecture

IIT Delhi – Abu Dhabi
Semester II, 2025–26

Assignment 2: Design and Implementation of a Microprogrammed 4-Bit Processor
Due Date: **May 10, 2026**

Total Marks: 50

---

1. Objective

The goal of this assignment is to design, simulate, and test a complete 4-bit microprocessor using the Logisim-Evolution digital logic simulator. You will build the datapath from fundamental logic gates, implement a specific Instruction Set Architecture (ISA) that includes loops and conditional branching, and design a **Microprogrammed Control Unit** to execute these instructions via sequential micro-operations.

2. Logisim

Download hardware simulator environment **Logisim** using Link.

3. Rules and Constraints

To ensure a rigorous understanding of the underlying digital logic within processor components, the following design constraints apply to your implementation:

- **The "Build It Once" Rule:** You **may not** use Logisim's built-in Multiplexers, Decoders, or ALU components in your final circuit.
- **Basic Gates Only:** You must design your Multiplexers (e.g., 2-to-1, 4-to-1), Decoders (e.g., 2-to-4), and the complete Arithmetic Logic Unit (ALU) strictly using basic logic gates (AND, OR, NOT, NAND, NOR, XOR).
- **Subcircuits:** Once you have built a component (like a MUX or Decoder) from basic gates, you must save it as a Logisim Subcircuit. You may then instantiate and use your custom Subcircuits freely throughout the processor.
- **Allowed Built-in Components:** You are permitted to use Logisim's built-in `RAM` and `ROM` modules for memory, built-in `Registers` for the Register File, as well as `Splitters`, `Tunnels`, `Pins`, and the `Clock`.

4. Architecture Specifications

You will implement a **Harvard Architecture** (utilizing separate instruction and data memories) to accommodate the addressing limitations of a 4-bit datapath.

- **Datapath & Data Memory:** 4-bit wide. All calculations, general-purpose registers, and the Data RAM operate on 4-bit words.
- **Register File:** Four 4-bit General Purpose Registers (`R0`, `R1`, `R2`, `R3`).
- **Instruction Memory (ROM):** 8-bit wide.
- **Program Counter (PC):** 8-bit wide.
- **Flags Register:** A 1-bit register that stores the **Zero Flag (Z)**. This flag is updated by the ALU whenever an arithmetic or logical operation results in `0000`.

5. The Instruction Set Architecture (ISA)

Your processor must support the following 9 instructions. Each instruction is 8 bits long, structured as `[Opcode: 4 bits] [Operand: 4 bits]`. The processor supports 2's complement substraction.

Opcode	Mnemonic	Format	Description	Updates Z?
`0001`	**LDI**	`0001 [4-bit Imm]`	**Load Immediate:** Load the 4-bit immediate value into `R0`.	No
`0010`	**ADD**	`0010 [Reg A] [Reg B]`	**Add:** `Reg A` ← `Reg A` + `Reg B`.	Yes
`0011`	**SUB**	`0011 [Reg A] [Reg B]`	**Subtract:** `Reg A` ← `Reg A` - `Reg B`.	Yes
`0100`	**AND**	`0100 [Reg A] [Reg B]`	**Bitwise AND:** `Reg A` ← `Reg A` AND `Reg B`.	Yes
`0101`	**LD**	`0101 [4-bit Addr]`	**Load:** Read data from Data RAM at `Addr` and store it in `R0`.	No
`0110`	**ST**	`0110 [4-bit Addr]`	**Store:** Write the contents of `R0` into Data RAM at `Addr`.	No
`0111`	**JMP**	`0111 [4-bit Addr]`	**Jump:** Unconditionally change the PC to the 4-bit `Addr`.	No
`1000`	**JZ**	`1000 [4-bit Addr]`	**Jump if Zero:** Change the PC to `Addr` ONLY if the Zero Flag (Z) is 1.	No
`1001`	**DEC**	`1001 [Reg A] 00`	**Decrement:** Subtract 1 from `Reg A`.	Yes

6. The Microprogrammed Control Unit

You must implement a microprogrammed control unit using a ROM module. Hardwired control logic is **not allowed**.

6.1. Control ROM Implementation

1. **The Step Counter:** Utilize a 2-bit or 3-bit counter that increments on every clock cycle to track the execution sequence (e.g., Fetch, Decode, Execute).
2. **Addressing the Control ROM:** The address input to your Control ROM will be a concatenation of the **Instruction Opcode** (from the Instruction Register) and the **Step Counter**.
3. **The Control Word:** The output of this ROM is your wide Control Word. You must use a Splitter component to break this word into individual wires to dictate the control signals of the datapath (e.g., `Mem_Read`, `Reg_Write`, `ALU_Opcode`).

6.2. Implementing the Conditional Jump (JZ)

To facilitate the `JZ` instruction, your Control Word must allocate a specific control bit, for example, `Branch_If_Zero`. During the execute phase of a `JZ` instruction, the microcode should assert `Branch_If_Zero = 1`. In your datapath, use an AND gate to combine the `Branch_If_Zero` signal with the output of your **Zero Flag Register**. Route the output of this AND gate to the 'Load' pin of your Program Counter.

7. Deliverables and Submission

Please submit a single ZIP file containing the following items:

1. `processor.circ`**:** Your complete Logisim-Evolution project file. Ensure all subcircuits are logically named and neatly organized.
2. **Microcode Spreadsheet:** A table (PDF or Excel format) documenting your Control Word layout. You must list every utilized address in your Control ROM, the binary sequence stored at that address, and a plain-English description of the micro-operations occurring at that step.
3. **Test Program (**`loop_test.hex`**):** A small assembly program, converted to machine code hexadecimal, loaded into your Instruction ROM that demonstrates a working loop.
    - *Requirement:* Your program must load the value `4` into `R0`. It must then execute a loop utilizing the `DEC` and `JZ` instructions to count down to `0`. Once `R0` reaches zero, the program must exit the loop and store a final value into the Data RAM.